Proyecto I: Procesador de Imágenes Erlang↔Scheme

IC-4700 Lenguajes de Programación

TEC | Tecnológico de Costa Rica

Estudiante:
Josué Monterrey Hernández

Carné:
2025071773

Profesor:
Eddy Ramírez Jiménez

17 de septiembre de 2026

Índice

1. Arquitectura general **1**

2. División de la imagen **2**

3. Comunicación entre procesos **3**
3.1. Modelo de concurrencia 3
3.2. Comunicación Erlang–Scheme 3
3.3. Protocolo de comunicación 3

4. Detalles de implementación **4**
4.1. Implementación del filtro 4
4.2. Manejo de bordes 4
4.3. Manejo de errores 4

5. Resultados experimentales **5**

6. Evaluación de rendimiento y análisis de eficiencia **8**
6.1. Filtro - Escala de Grises 9
6.2. Filtro - Gaussian Blur 10
6.3. Análisis y comparación 11

1. Arquitectura general

El sistema es controlado por un script principal escrito en Erlang (`main.erl`) que se encarga de:

1. Leer la imagen de entrada, en formato PPM tipo P3.
2. Extraer las dimensiones de la imagen y los valores de cada pixel.
3. Crear múltiples procesos paralelos quienes toman una sección distinta de la imagen.
4. Enviar cada sección a un script externo de Scheme que le aplica el filtro deseado a cada pixel.
5. Recibir los nuevos pixeles modificados y reconstruir la imagen.

Existe un script base en Scheme (`filtro_base.scm`) que contiene la lógica para recibir la sección de una imagen enviada desde Erlang y aplicarle una función filtro a cada pixel, además de enviar la sección modificada de vuelta a `main.erl`.

Por último, cada filtro es aplicado desde otro script de Scheme externo que implementa una función que modifica el valor de un único pixel. De esta manera es muy sencillo crear nuevos filtros o modificar los actuales.



Figura 1: Diagrama general de la arquitectura del sistema

2. División de la imagen

La imagen es interpretada como una matriz (lista de listas) donde cada valor individual de la matriz es un pixel, que a su vez se representa como una lista con tres elementos (rojo, verde y azul).

La imagen es dividida en franjas horizontales cuya cantidad de filas se divide equitativamente entre la cantidad de procesos paralelos estipulados por el usuario. En caso de que la imagen no tenga una altura divisible por la cantidad de procesos, cada sección generada tendrá una linea extra hasta agotar la cantidad de lineas sobrantes.

Cabe destacar que cada sección lleva consigo una cantidad variable de lineas extra que no forman parte del resultado final que se envía de vuelta, pero que sirven como "contexto" en casos donde el valor de un pixel depende de sus vecinos. Esta cantidad es indicada por el usuario como parte de los parametros de la funcion principal.

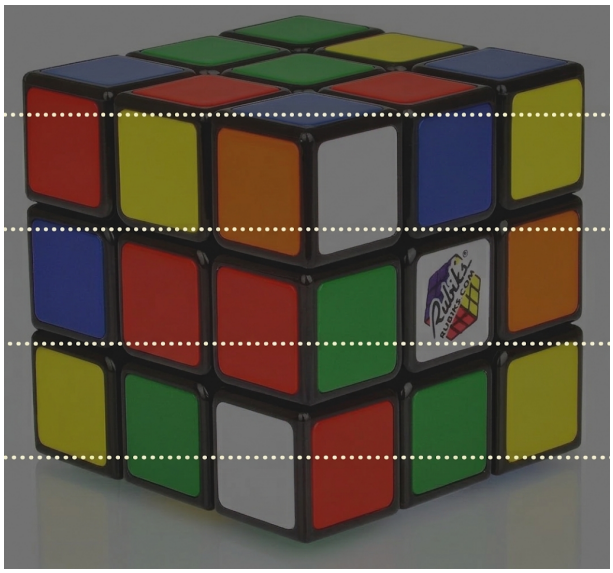


Figura 2: Ejemplo de la división de imágenes

3. Comunicación entre procesos

3.1. Modelo de concurrencia

- Se hace `spawn` de un proceso nuevo por cada sección de imagen generada.
- Cada proceso abre un puerto hacia el script de Scheme para que procese su sección y envía una respuesta al proceso padre.
- El proceso padre espera a recibir una respuesta de todos los procesos y reconstruye la imagen.

3.2. Comunicación Erlang–Scheme

- Se utiliza un modelo en envío de mensajes por consola (`stdin`/`stdout`), sin memoria compartida. Primeramente, Erlang envía una sección completa a Scheme, el filtro la lee y la procesa y posteriormente la nueva sección es enviada completa a través del mismo medio.

- Se utiliza el siguiente comando que permite pasar como argumento de la función el script del filtro especifico que se desea aplicar:

```
1 Port=open_port({spawn, "racket "++Filtro}, [stream, {line, 1024}])
```

- Otro punto importante a destacar es que cada sección es enviada al filtro junto con un identificador (`ID`) que se utiliza en la fase de reconstrucción para unir cada pieza de la imagen en el orden correcto.

3.3. Protocolo de comunicación

El protocolo es texto plano, línea por línea para ambos lados (Erlang y Scheme).

1. La primera linea que espera el filtro representa el **ancho** de la sección.
2. La segunda es el **alto**, es decir, la cantidad total de filas enviadas incluyendo el contexto.
3. Luego, se espera una línea por cada **píxel** de la sección, en orden, fila por fila y con el formato "`R G B`".

4. Detalles de implementación

4.1. Implementación del filtro

El archivo `filtro_base.scm` tienen las bases para recibir la sección enviada desde Erlang y también para enviar de vuelta el resultado del filtro. Este script expone la función (`procesar`) que delega la transformación pixel por pixel a una función externa que deberá proveer el script del filtro como tal.

4.2. Manejo de bordes

Si se requiriera procesar pixeles fuera de los bordes laterales o si se agotaran las filas de contexto el resto de pixeles se interpretan como pixeles negros `(0.0 0.0 0.0)`.

Esto permite ver una limitación del sistema que es importante a tener en cuenta: el tamaño de cualquier *kernel* no puede ser mayor al de las lineas de contexto para evitar cálculos erróneos.

4.3. Manejo de errores

- El sistema no cuenta con validación para el archivo de entrada ya que espera una imagen con extensión `.PPM` del tipo P3 con valores de pixeles enteros y separados por espacios y saltos de línea.
- Tampoco se cuenta con validación sobre la existencia del archivo de la imagen o del filtro elegido, se asume que sí existe un archivo con ese nombre en ese directorio.
- No se validan los valores resultantes de aplicar el filtro así que el sistema fallaría si algún valor no fuera un número entero.

5. Resultados experimentales

Para propósitos de demostración se utilizó la siguiente fotografía de un cubo de Rubik tomada de *https://www.ebay.com/itm/291758021771*:



Figura 3: Imagen sin filtro de un cubo de Rubik

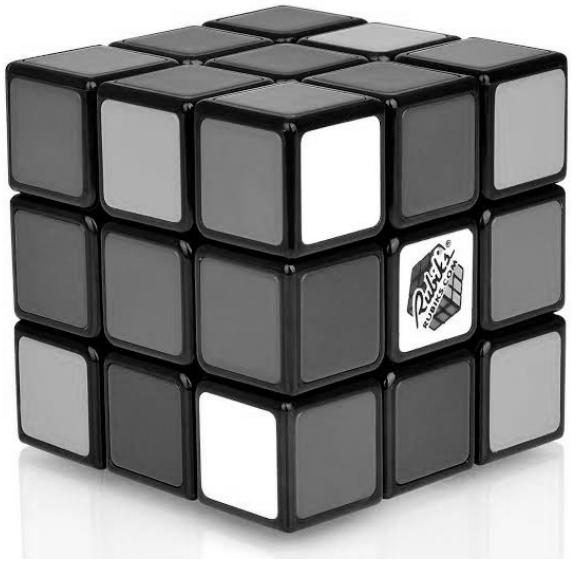


Figura 4: Filtro de escala de grises

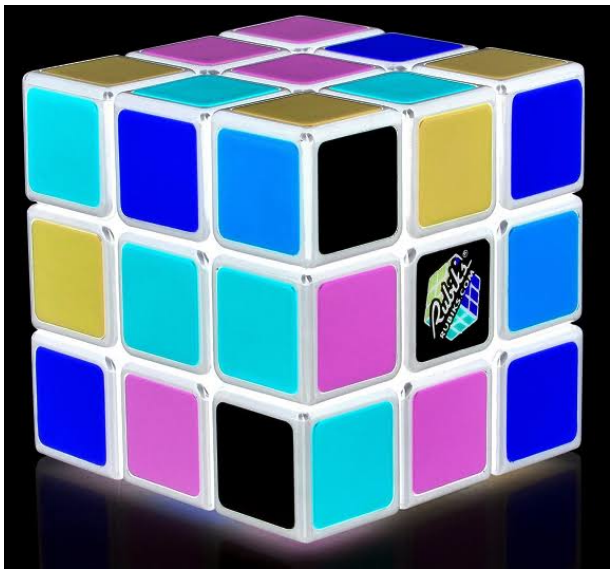


Figura 5: Filtro de inversión de colores

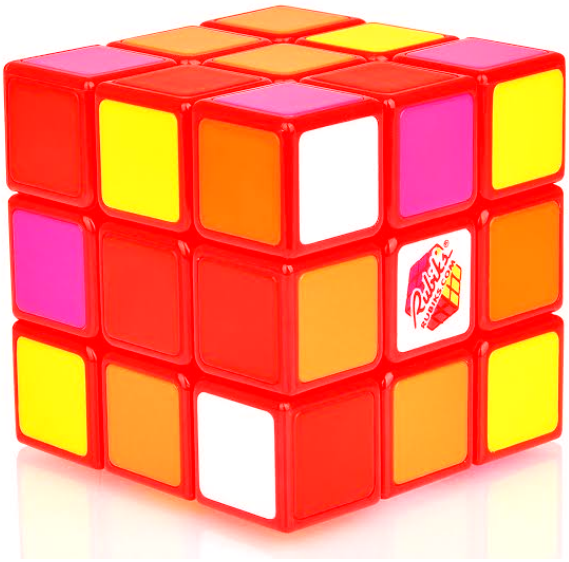


Figura 6: Filtro donde todos los pixeles tiene el valor R=255



Figura 7: Filtro con *gaussian blur* ligero



Figura 8: Filtro con *gaussian blur* fuerte

6. Evaluación de rendimiento y análisis de eficiencia

Para las diferentes pruebas de rendimiento se utilizó la misma imagen de cubo Rubik que se ha mostrado durante todo el documento. Esta imagen tiene un alto de 574 pixeles y un ancho de 534 pixeles. Se realizaron pruebas con dos filtros distintos: desenfoque gaussiano y escala de grises. Esto ya que el filtro de escala de grises solamente procesa un pixel a la vez mientras que el otro filtro toma en cuenta múltiples pixeles vecinos del central.

Cada prueba consiste en 5 mediciones de tiempo con cada filtro utilizando desde 1 proceso hasta 8 procesos paralelos. Los tiempos de estas 5 mediciones fueron promediados para obtener el tiempo final.

A continuación se detallan algunas características importantes sobre el entorno en el cual se realizaron las pruebas:

- CPU: Intel core i5 10th gen
  - Núcleos: 4
  - Hilos: 8
  - Velocidad: 1.00 GHz
- RAM: 16 GB
- SO: Linux Mint Cinnamon

6.1. Filtro - Escala de Grises

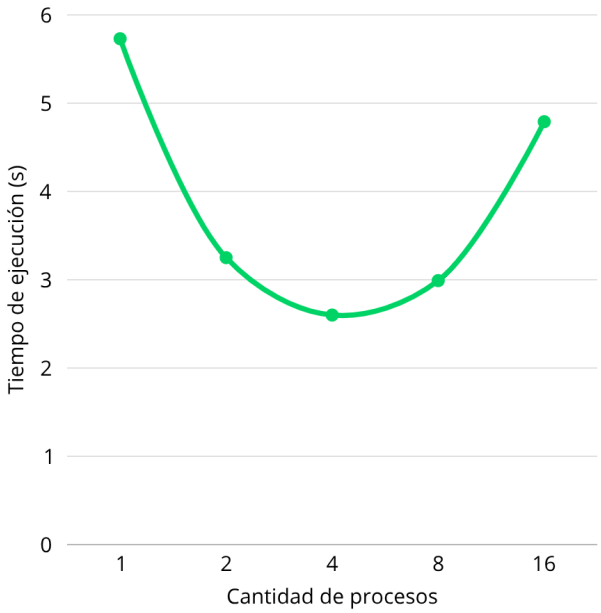


Figura 9: Cantidad de procesos vs tiempo de ejecución promedio

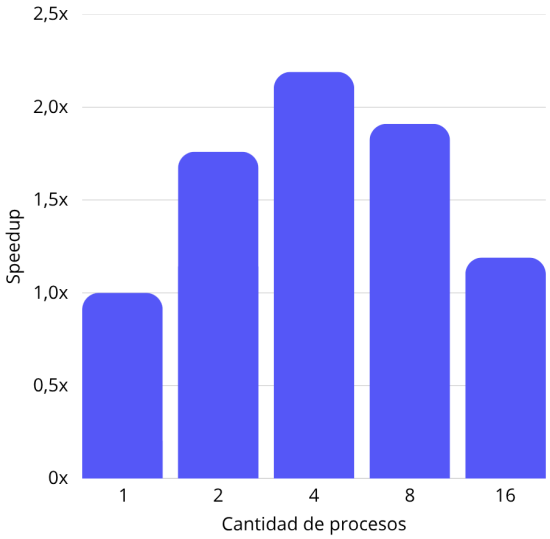


Figura 10: Gráfico de speedup

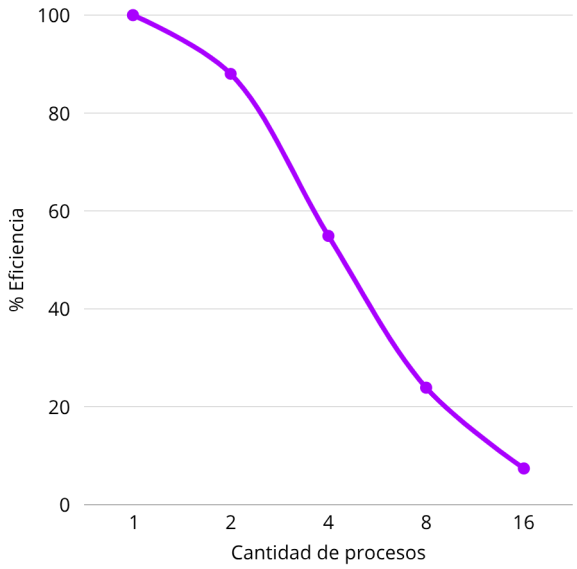


Figura 11: Gráfico de eficiencia

# Procesos	Tiempo promedio (ms)	Speedup	Eficiencia
1	5732	1x	100 %
2	3255	1,76x	88,05 %
4	2608	2,19x	54,95 %
8	2996	1,91x	23,92 %
16	4798	1,19x	7,47 %

6.2. Filtro - Gaussian Blur

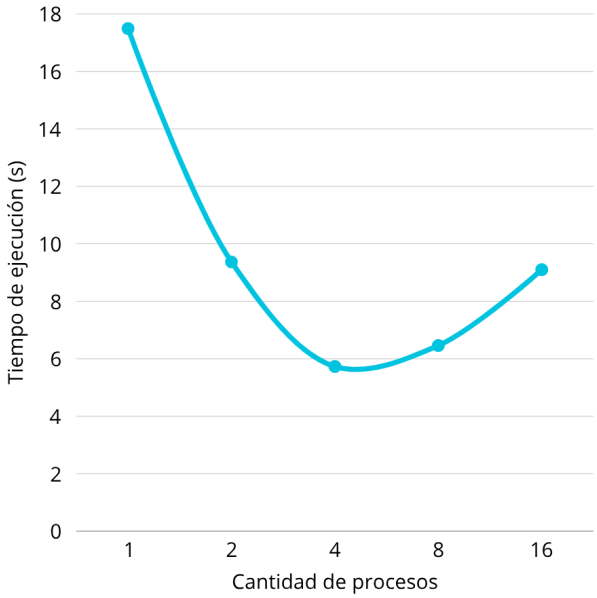


Figura 12: Cantidad de procesos vs tiempo de ejecución promedio

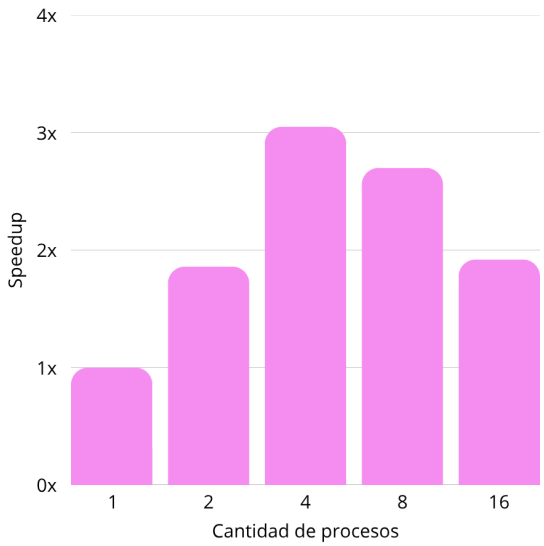


Figura 13: Gráfico de speedup

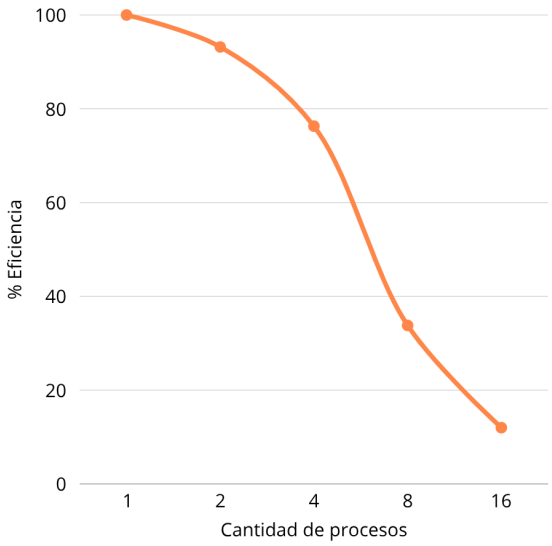


Figura 14: Gráfico de eficiencia

Procesos	Tiempo promedio (ms)	Speedup	Eficiencia
1	17493	1x	100 %
2	9379	1,86x	93,26 %
4	5731	3,05x	76,31 %
8	6463	2,70x	33,83 %
16	9109	1,92x	12 %

6.3. Análisis y comparación

A continuación se realiza un análisis comparativo entre los tiempos de ejecución, speedup y eficiencia de ambos filtros de prueba.

Tiempo de ejecución base: El filtro de desenfoque gaussiano tiene un tiempo base de 17,5 segundos, más de tres veces mayor que el tiempo base de escala de grises (5,7s). Esto es consistente con la manera de funcionar de cada algoritmo: la escala de grises aplica una única operación por píxel (calcular el promedio de sus canales RGB), mientras que el desenfoque gaussiano requiere una convolución con un kernel, es decir, múltiples operaciones de lectura y multiplicación por cada píxel de salida, además de la creación y normalización previa de la matriz gaussiana.

Mejoras con paralelización: En ambos filtros el speedup máximo se alcanza con 4 procesos (2,19x en escala de grises y 3,05x con gaussian), y a partir de ese punto el rendimiento empeora con 8 y 16 procesos. Este comportamiento es el esperado ya que como se especificó anteriormente, las pruebas se realizaron en una computadora con 4 núcleos físicos y al superar ese número, el trabajo para la sincronización, creación de hilos y manejo de memoria supera la ganancia de paralelizar el procesamiento.

Eficiencia: El gaussian blur tiene una eficiencia más alta en general que la escala de grises en cada cantidad de procesos:

# Procesos	Desenfoque gaussiano	Escala grises
2	93,26 %	88 %
4	76,31 %	55 %
8	33,83 %	23,9 %
16	12 %	7,5 %

Esta diferencia se explica por las distintas operaciones que realiza cada filtro. Al ser el gaussian blur un algoritmo más pesado por píxel, el tiempo que cada proceso dedica a trabajo útil es mucho mayor en relación con el tiempo fijo de separar datos, sincronización y envío de resultados. Por otro lado, el filtro de escala de grises realiza una operación muy sencilla y por ende ese tiempo relativo pesa más rápido a medida que se agregan procesos.

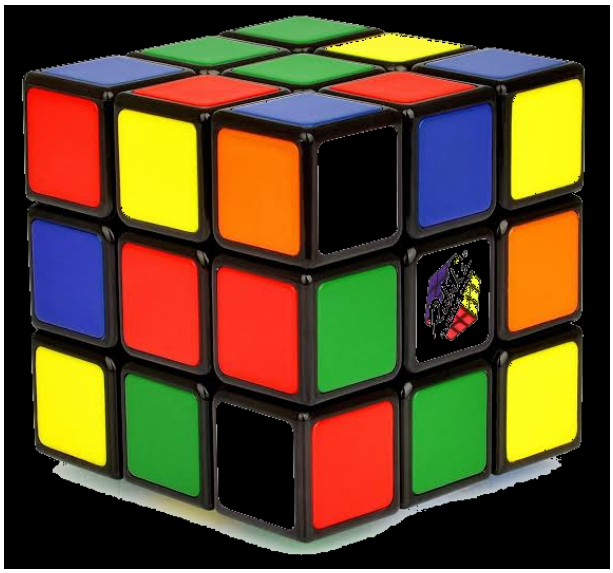


Figura 15: Bonus: un filtro que toma todos los pixeles cercanos al blanco y los convierte en negros.